

AI Full-Stack Trainer: 60-Minute Course Recap Handbook

Quick navigation index

1. Purpose
2. Learning outcomes
3. Exact 60-minute clock
4. Before the session
5. 60-minute trainer runbook
 - 00:00–04:00 — 1–2: Foundations and setup
 - 04:00–12:00 — 3–8: Encodings and embeddings
 - 12:00–26:00 — 9–15: Transformer architecture
 - 26:00–34:00 — 16–17: APIs and multimodality
 - 34:00–47:00 — 18–28: Fine-tuning and alignment
 - 47:00–58:00 — 29–31: RAG
 - 58:00–60:00 — Full-stack close
6. Class-by-class coverage guide
7. Trainer reference sheets
 - Representation decision guide
 - Model adaptation decision guide
 - Core formula board
 - RAG minimal pseudocode
8. AI full-stack trainer bridge
 - Reusable classroom case study
 - Five-question design review
9. Delivery checks and common misconceptions
10. Final trainer preparation checklist

Purpose

This handbook turns the material from Class 1 through Class 31 into one trainer-led, 60-minute recap. It is designed for teaching, not silent reading: each section tells you what to explain, what to draw, which original classroom example to reuse, what to ask learners, and how to transition to the next topic.

The recap follows the course sequence and uses examples found in the class notes, notebooks, PDFs, and code. Assignment material is outside the scope of this handbook. Class 6–7 and Class 16–17 are combined in the workspace, so they remain combined here.

Your trainer goal is larger than remembering definitions. By the end, you should be able to explain the complete journey:

raw data → numerical representation → transformer → foundation model → API or fine-tuned model → retrieval → backend → user-facing AI application

What a learner should be able to explain after the hour

- Why generative AI is different from traditional predictive AI.
- Why text must be converted into numbers and how OHE, BoW, TF-IDF, Word2Vec, and transformer embeddings differ.
- How a transformer uses positional information, self-attention, multi-head attention, residual connections, normalization, and feed-forward layers.
- When to use an encoder, decoder, encoder-decoder model, large language model, small language model, or multimodal model.
- How to access hosted and local models through SDKs and LangChain abstractions.
- What changes during pretraining, continued pretraining, instruction tuning, and preference alignment.
- When full fine-tuning, LoRA, QLoRA, Unsloth, managed fine-tuning, RAG, or a combination is appropriate.
- How a RAG system parses, chunks, embeds, retrieves, augments, and generates a grounded response.
- Where the AI pipeline fits inside a full-stack application.

The exact 60-minute clock

Time	Duration	Course coverage	Teaching objective
00:00–04:00	4 min	1–2	Set up the AI journey and the product mindset
04:00–12:00	8 min	3–8	Move from text encoding to semantic embeddings
12:00–26:00	14 min	9–15	Build transformer intuition from input to output
26:00–34:00	8 min	16–17	Show model access, multimodality, and application integration
34:00–47:00	13 min	18–28	Explain fine-tuning, PEFT, LoRA/QLoRA, and alignment
47:00–58:00	11 min	29–31	Build the end-to-end RAG mental model
58:00–60:00	2 min	Whole course	Close with the AI full-stack architecture and teach-back
Total	60 min	Class 1 through Class 31	One connected story

The clock applies to the main runbook only. The later class index, decision guides, formulas, and checklists are preparation and follow-up references. The timed sections contain more wording than you should read aloud; rehearse the explanation in your own voice, demonstrate only the nominated examples, and use the timer to preserve each transition.

Before the session

Prepare only a small number of reliable visuals and demos. A recap fails when the trainer spends the hour waiting for models to download.

- Open this handbook and keep a visible timer.
- Prepare one blank slide or whiteboard for the end-to-end architecture.
- Pre-run the small local examples you intend to show.
- Keep screenshots for API, GPU, or large-model notebooks as fallbacks.

- Load API keys through environment variables; never paste keys into a notebook or slide.
- Re-run a notebook top to bottom before presenting it. Some saved notebook outputs were produced in earlier executions and do not always match the latest source cell.
- Treat model names, API capabilities, free-tier limits, and SDK syntax in the class notebooks as point-in-time examples. Verify them before a live external demo.
- Avoid downloading large embedding or language models during the recap.

Use a repeating trainer rhythm throughout the hour:

1. **Explain** the problem in plain language.
 2. **Draw** the smallest useful mental model.
 3. **Demonstrate** one course example.
 4. **Ask** one diagnostic question.
 5. **Connect** the answer to the next layer.
-

The 60-minute trainer runbook

00:00–04:00 — 1–2: Foundations, setup, and the product journey

Teaching outcome

Learners should understand where generative AI sits in the broader AI landscape and why the course starts with Python tooling but ends with applications.

Say

"Artificial intelligence is the broad field. Machine learning learns patterns from data. Deep learning uses neural networks to learn complex representations. Generative AI goes one step further: instead of only predicting a class or a number, it learns patterns in the data well enough to produce new text, images, audio, video, or code."

Draw this hierarchy:

```
Artificial Intelligence
├─ Machine Learning
│   └─ Deep Learning
│       └─ Generative AI
│           ├── Foundation models
│           ├── Retrieval-augmented systems
│           └─ Tool-using agents
```

Then distinguish three application behaviors:

- **Predictive AI:** "Is this transaction fraudulent?"
- **Generative AI:** "Write an explanation of why the transaction looks suspicious."
- **Agentic AI:** "Investigate it, query the transaction system, prepare a report, and ask for approval before acting."

An agent therefore needs more than text generation. It needs an objective, context, reasoning or decision logic, tools, state, and boundaries.

Demonstrate the original Python starting point

Use the course `Calculator` class as the smallest example of a dependable software component:

```
class Calculator:
    def add(self, a, b):
        return a + b

    def divide(self, a, b):
        if b == 0:
            return "Cannot divide by zero"
        return a / b
```

Explain why it belongs in an AI course: an AI application is still software. The model may decide that division is needed, but a deterministic tool should perform the arithmetic. This is the first bridge from Python fundamentals to tool-using agents.

Mention the development workflow

The setup lessons introduced `uv`, Python, virtual environments, and reproducible dependencies. Keep the recap concise:

```
winget install uv
uv python install 3.12
uv init
uv add <package-name>
uv run python <script-name>.py
```

The key teaching point is not command memorization. It is reproducibility: source code, Python version, dependencies, environment variables, and execution command should be understandable to another developer.

Ask

"If an LLM can calculate, why still give it a calculator tool?"

Expected answer: language models generate likely tokens and can make arithmetic errors; tools provide deterministic, testable behavior.

Transition

"A model cannot consume the sentence `I love AI` directly. Before it can reason or generate, language has to become numbers."

04:00–12:00 — 3–8: From sparse encodings to semantic embeddings

Teaching outcome

Learners should see representation learning as an evolution, with each method solving a limitation of the previous one.

04:00–05:30 — Encoding versus embedding

Say:

"Encoding is the broad act of converting information into a numerical form. An embedding is a learned, usually dense vector designed so that useful meaning or relationships are represented geometrically."

Quick comparison:

Method	Representation	Meaning captured?	Context-sensitive?	Typical limitation
One-hot encoding	Sparse binary vector	No	No	Large vocabulary and no similarity
Bag of Words	Sparse count vector	Very little	No	Loses order and meaning
TF-IDF	Sparse weighted vector	Limited	No	Still lexical and vocabulary-bound
Word2Vec	Dense learned word vector	Yes	No	One vector per word sense
Transformer embedding	Dense learned vector	Yes	Yes	More compute, model, and cost choices

Explain vector intuition: a scalar is one number, a vector has magnitude and direction, and a matrix is an organized collection of vectors. Similar meanings should occupy nearby positions in an embedding space.

05:30–08:00 — Reuse the course encoding examples

One-hot encoding

Use the source sentence:

```
document = ["my name is sunny and I love AI"]
```

Tokenize it, build a vocabulary, and explain that each word receives a separate axis. **AI** and **love** are no closer than **AI** and **name**; every distinct category is orthogonal.

Bag of Words

Use the course corpus:

```
documents = [  
    "people watch movie and watch movie again",  
    "people watch cricket and watch cricket",  
    "people like movie and like movie a lot",  
    "people like cricket",  
]
```

Ask learners what BoW preserves. It keeps vocabulary counts: the first document has repeated **watch** and **movie**. Ask what it loses. It loses word order; it does not understand that cricket and movie are different activities.

Use the out-of-vocabulary sentence **lion is the king of jungle**. If those terms were not in the fitted vocabulary, the vectorizer cannot give them useful dimensions.

TF-IDF

Reuse the same corpus. Explain that words appearing frequently in one document are important, but words appearing in nearly every document should receive less discriminating weight.

```
TF-IDF(term, document) = term frequency × inverse document frequency
```

Do not get trapped in library-specific smoothing during a recap. Teach the intuition first: **people** appears broadly, while a less common word can better distinguish a document.

08:00–10:00 — Word2Vec and static meaning

Explain the two learning views:

- **CBOW**: use surrounding words to predict the center word.
- **Skip-gram**: use a center word to predict surrounding words.

Use the course's pretrained Word2Vec demonstrations:

```
king - man + woman ≈ queen  
odd one out: PHP, JAVA, DOG, C++ → DOG  
similarity(man, woman) > similarity(man, python)
```

Then expose the limitation with the exact **bank** example:

```
I love to sit near river bank.  
I always deposit money in bank.
```

Static Word2Vec supplies one vector for `bank`, even though the meanings differ. Averaging the four word vectors in `I love machine learning` produces a simple sentence vector, but averaging can wash out order and nuance.

The custom Word2Vec lesson trained on two small stories about Arjun and AI. The noisy similarities are an important teaching result: neural embeddings are data hungry. A tiny, narrow corpus cannot magically produce a universal semantic map.

10:00–12:00 — Contextual and multimodal embeddings

Use `SentenceTransformer("all-MiniLM-L6-v2")` to show that a word, a sentence, or a paragraph can become a fixed-size vector. In the course notebook the vectors have 384 dimensions. Other notebook examples used hosted Gemini and OpenAI embedding models, while CLIP produced an image embedding.

Use the heart-risk semantic-search example:

```
Query: How to reduce heart disease risk?  
  
Documents:  
1. Eating fiber reduces heart risk.  
2. Fruits and vegetables lower cardiovascular disease chances.  
3. Buying a new car improves driving comfort.  
4. Regular exercise improves heart health.
```

A keyword matcher misses some wording changes. An embedding search recognizes that `cardiovascular disease` relates to `heart disease`, while the new-car sentence is semantically distant.

Draw:

```
query —embed→ vector —similarity→ document vectors —rank→ top result
```

Name the measures without overloading the audience:

- Dot product: larger generally means more alignment, but magnitude matters.
- Cosine similarity: compares direction; closer to 1 means more similar.
- Euclidean distance: smaller means closer.

Mention MTEB as a way to compare embedding models across tasks, but choose a model based on language, domain, quality, latency, dimensions, and cost—not leaderboard position alone.

Ask

"Which representation would you choose for semantic search over differently worded questions, and why?"

Expected answer: contextual sentence/document embeddings because they encode meaning beyond exact token overlap.

Transition

"Embeddings provide useful vectors. A transformer explains how those vectors become contextual and how modern models process relationships across a sequence."

12:00–26:00 — 9–15: Transformer intuition and architecture

Teaching outcome

Learners should be able to trace information through a transformer and explain the responsibility of each major block.

12:00–14:00 — Why transformers replaced sequence bottlenecks

Start with the earlier sequence-model limitation:

- RNNs process tokens one after another, limiting parallel training.
- Long dependencies are difficult to preserve through many sequential steps.
- LSTMs improve memory but retain sequential computation.
- Attention lets a model focus directly on relevant positions.
- Transformers make attention the central operation and process input tokens in parallel during training.

Add one nuance expected from a trainer: transformer training can parallelize known sequence positions, but autoregressive generation still emits the next token step by step.

Draw the main path:

```
tokens
↓
token embeddings + positional encoding
↓
multi-head self-attention
↓
Add & Norm
↓
feed-forward network
↓
Add & Norm
↓
contextual representations
```

14:00–16:00 — Position and order

Use the trainer's examples:

```
I love AI      vs.      AI love I
dog bites man  vs.      man bites dog
```

Without position information, a parallel set of word embeddings does not carry sequence order. The course introduces sinusoidal positional encoding:

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

The important verbal explanation is: **pos** identifies token position, **i** identifies a dimension pair, and different frequencies create a unique, smoothly related position pattern. Positional vectors are **added** to token embeddings, so the model dimension stays unchanged.

16:00–19:00 — Self-attention with Q, K, and V

Use the course's plain-language meanings:

- **Query:** What is this token looking for?
- **Key:** What information can this token match on?
- **Value:** What content should be passed forward if it is relevant?

Write the core equation:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Walk it slowly:

1. Project each input through learned W_q , W_k , and W_v matrices.
2. Compute query-key similarity scores.
3. Divide by $\sqrt{d_k}$ to keep large dot products from making softmax too sharp.
4. Apply softmax to turn scores into attention weights.
5. Compute a weighted combination of values.

Use **money bank grows** and **river bank flows**. The **bank** query sees different relevant keys in the two contexts, so its output representation changes.

Use the second course example:

Apple launched a new phone while I was eating the apple.

The first **Apple** should attend to company/product words; the second **apple** should attend to eating. This is the bridge from Class 8's contextual embeddings to the transformer mechanism.

19:00–21:00 — Multi-head attention, residual paths, normalization, and FFN

Multi-head attention

One head provides one learned view of relationships. Multiple heads can attend to syntax, entity relationships, distance, or other useful patterns in parallel.

Use the ambiguity examples:

The chicken is ready to eat.
The man saw the astronomer with a telescope.

Different heads can represent competing interpretations and contextual relationships.

Residual connection

Explain $\text{output} = \text{input} + \text{transformation}(\text{input})$. The original signal has a direct path, gradients flow more easily, and each block can learn a useful refinement rather than reconstructing everything.

Layer normalization

The course normalizes an example token vector such as $[0.8, -1.2, 2.4]$, then applies learned scale γ and shift β . Teach the purpose: stable feature scale and more reliable training. Layer normalization operates across the features of each token, unlike batch normalization's dependence on batch statistics.

Feed-forward network

Self-attention exchanges information among tokens. The feed-forward network then applies the same nonlinear transformation independently to every token position. A simple distinction learners can retain is:

Attention mixes information across tokens; the FFN transforms features within each token.

Stacking blocks repeatedly builds richer representations. The number of layers is a model design choice; it is not fixed at the original Transformer paper's configuration.

21:00–24:00 — Encoder, decoder, masks, and cross-attention

Draw the three families:

Architecture	Main strength	Typical use
Encoder-only	Bidirectional understanding	classification, extraction, embeddings
Decoder-only	Autoregressive generation	chat, completion, code generation
Encoder-decoder	Conditioned sequence transformation	translation, summarization

For the decoder, distinguish two attention operations:

- **Masked self-attention:** the current target position cannot see future target tokens.
- **Cross-attention:** decoder queries attend to encoder keys and values.

Reuse the translation example `we are friends` → Hindi. The encoder represents the English sentence. The decoder generates the translated sentence one token at a time while cross-attending to the relevant English representations.

Explain shifted targets and causal masking:

```
decoder input: <START> we are
target output: we      are friends
```

During training, the full target is known and masked positions can be computed in parallel. During inference, each generated token becomes input to the next step. A final linear projection and softmax produce next-token probabilities.

24:00–26:00 — Connect the concepts to the from-scratch transformer

The Class 15 notebook implements a small German-to-English transformer using the Multi30k dataset. Use it as a map, not as a live training demo:

```
spaCy tokenization
→ source and target vocabularies
→ PAD/SOS/EOS/UNK token IDs
→ embeddings + positional encoding
→ encoder and decoder layers
→ source padding mask + target causal mask
→ cross-entropy loss ignoring PAD
→ teacher-forced training
→ greedy translation
```

The notebook’s `PositionalEncoding`, `MultiHeadAttention`, `EncoderLayer`, `DecoderLayer`, and `Transformer` components are concrete implementations of lessons 9–14. Its small configuration—such as 256 model dimensions, 8 heads, and 3 layers—is educational, not a production translation system.

Close the section by locating model categories:

- **LLM:** large, broadly capable language model, typically hosted because of compute needs.
- **SLM:** smaller model that can reduce latency and cost or run closer to the user.
- **Multimodal LLM:** accepts or generates more than text, such as images, audio, or video.

Ask

“What does self-attention do that a feed-forward layer does not?”

Expected answer: self-attention combines information across token positions; the FFN nonlinearly transforms each token’s features.

Transition

“We now understand the engine. The next question is how a real application calls that engine and handles different input and output modes.”

26:00–34:00 — 16–17: APIs, LangChain, local models, and multimodality

Teaching outcome

Learners should understand the provider abstraction and be able to place a model call inside an application flow.

26:00–28:00 — Ways to access a model

Present the spectrum from the class:

Access pattern	Examples used in class	Why choose it	Main trade-off
Provider API	OpenAI, Gemini, Groq, OpenRouter, Hugging Face	Fastest integration	Network, pricing, limits, data policy
Framework wrapper	LangChain chat-model integrations	Common message and chaining interface	Another abstraction to understand
Direct SDK	Provider client libraries	Full provider feature access	Provider-specific code
Local runtime	Ollama	Privacy, offline use, control	Local hardware and model operations
Managed cloud endpoint	Cloud model services	Governance and scalable operations	Cloud configuration and cost

Explain that LangChain does not create the model. It standardizes integration concepts such as system messages, human messages, prompts, tools, retrievers, and runnable chains. Use the direct SDK if the wrapper does not expose a new provider capability.

28:00–31:00 — Reuse the course’s compact API demonstrations

Pick two, not every provider, for a live recap:

1. Ask an OpenAI-compatible chat model: “What is the capital of India?”

2. Ask a Gemini- or Groq-backed model to translate “I love programming” into French or Hindi.

Explain the message roles:

```
messages = [  
    ("system", "You are a concise language tutor."),  
    ("human", "Translate 'I love programming' into Hindi."),  
]  
response = model.invoke(messages)
```

The system message establishes behavior; the human message carries the user’s request. The application should send only the context needed for the task and should handle timeouts, invalid credentials, rate limits, and unsafe output.

31:00–33:00 — Multimodal input and output

Use the course matrix:

```
text → text, image, or speech  
image → text  
audio → text
```

The lessons demonstrated text-to-image with a “cute fuzzy cat with an umbrella,” image captioning or OCR, audio transcription, text-to-speech, and local vision through Ollama-compatible models.

Explain that a multimodal request often contains a textual instruction plus encoded or uploaded media. The model still needs a clear task: “Describe this product image,” “Extract the visible text,” and “Identify defects” are different instructions for the same image.

33:00–34:00 — Place it inside an app

The Streamlit example allowed a user to choose provider, model, temperature, input mode, and output mode. Use it to teach separation of concerns:

```
UI selection  
→ validate input  
→ select provider adapter  
→ construct messages/media payload  
→ call model  
→ normalize response  
→ render text/image/audio
```

Do not teach temperature as an intelligence dial. It changes sampling randomness; factual tasks often benefit from lower variability, while creative tasks may tolerate more.

Ask

“When would you prefer a framework wrapper, and when would you call the provider SDK directly?”

Expected answer: use the wrapper for a portable, common application flow; use the provider SDK for unsupported or provider-specific features.

Transition

“Prompting changes the model’s immediate context. What if we need to change its domain knowledge, task behavior, or preferences more persistently?”

34:00–47:00 — 18–28: Fine-tuning, PEFT, multimodal tuning, and alignment

Teaching outcome

Learners should be able to select an adaptation method based on the desired change, available data, and compute budget.

34:00–36:00 — The adaptation ladder

Draw the course progression:

```

pretrained base model
|
├─ continued pretraining on raw domain text
|   teaches domain language and knowledge patterns
|
├─ supervised instruction tuning on input/output examples
|   teaches how to follow domain tasks
|
└─ preference alignment on prompt/chosen/rejected examples
    teaches which acceptable response is preferred
  
```

Make the central distinction:

- **Pretraining/continued pretraining data:** raw text or next-token sequences.
- **Instruction data:** prompt or instruction paired with a desired answer.
- **Preference data:** the same prompt paired with chosen and rejected answers.

Use the pharma example that runs through lessons 18 and 21–24. Raw pharmaceutical documents teach vocabulary and facts. An instruction such as “Explain the primary mechanism of metformin” with an accurate answer teaches response behavior. A chosen accurate answer versus a rejected claim that metformin primarily increases insulin secretion teaches preference.

36:00–38:30 — Hugging Face as the model-development workspace

Lessons 19–20 introduced three reusable layers:

- **Hub:** versioned repositories for models, datasets, tokenizers, and artifacts.
- **Datasets:** loading, mapping, filtering, splitting, sharding, and publishing data.
- **Transformers/tokenizers:** converting text to IDs and loading a base model or a task-specific head.

Clarify common `AutoModel` choices:

- `AutoModel` returns base hidden representations.
- `AutoModelForMaskedLM` adds a masked-token prediction head.
- `AutoModelForCausalLM` adds next-token generation behavior.

The tokenizer lessons used pretrained BERT/IndicBERT tokenizers and also trained a small BPE tokenizer. Teach tokenization as a model contract: the vocabulary, special tokens, normalization, and token IDs must match what the model expects.

38:30–41:00 — Full fine-tuning, LoRA, and QLoRA

Use this decision table:

Method	What is updated	Memory need	Best fit
Full fine-tuning	All or most base weights	Highest	Large budget and deep model adaptation
LoRA	Small low-rank adapter matrices	Lower	Efficient task/domain adaptation
QLoRA	LoRA adapters over a quantized frozen base	Lowest of these	Fine-tuning a larger model on limited GPU memory

Write the LoRA idea:

$$W' = W + (\alpha/r)BA$$

The pretrained matrix W remains frozen. Instead of learning a full update, LoRA learns two smaller matrices A and B . Rank r controls adapter capacity, and α/r scales the update. The notebooks targeted attention projections such as q , k , v , and o , and sometimes MLP projections such as `gate`, `up`, and `down`.

QLoRA stores the frozen base model in a low-bit representation—4-bit NF4 in the course examples—while training the small adapters with higher-precision computation. Quantization reduces memory; LoRA reduces the number of trainable parameters. They solve related but different resource problems.

41:00–43:00 — Three-stage pharma pipeline and Unsloth

Walk through the exact training story:

1. Extract and clean a pharma PDF.
2. Convert text into records, tokenize it, and pack causal-language-model blocks.
3. Continue training a TinyLlama base with QLoRA.
4. Merge or load the domain adapter as the starting point for instruction tuning.
5. Train on structured pharmaceutical instruction/response examples.
6. Add preference training on prompt/chosen/rejected records.
7. save adapters and, when appropriate, merge them into deployable model weights.

Class 24 showed this end to end with Unsloth and compared it with a standard Hugging Face PEFT workflow. The teaching message is not “one library always wins.” Unsloth optimizes supported model training for speed and memory; Hugging Face Transformers, PEFT, TRL, and Datasets provide the broader conceptual ecosystem.

Mention two training cautions:

- A one-epoch or tiny-row notebook is a learning demonstration, not evidence of production quality.
- A merged checkpoint is convenient for serving, while separate adapters are smaller and can support multiple task variants over one base model.

43:00–44:30 — Multimodal and managed fine-tuning

Vision-language tuning

Class 25 built a small iPhone image-caption dataset. Each record combined an image, an instruction such as “Describe this iPhone product image in one sentence,” and the desired text response. The dataset schema is as important as the model: preserve image content, conversational roles, and label alignment. The notebook used a supported vision-language model with vision-aware collation and LoRA modules.

Managed GPT fine-tuning

Class 26 showed the managed lifecycle using smartphone-support conversations:

```

prepare JSONL chat messages
→ validate examples
→ upload file with fine-tune purpose
→ create job against a supported base model
→ monitor status
→ call the resulting model
→ compare base, tuned, and ground-truth responses

```

Examples included a one-year limited warranty and different app availability on Android and iOS. Emphasize evaluation: a successful job status proves that training ran, not that the model became better.

44:30–47:00 — Preference alignment: RLHF, PPO, and DPO

Start from the need: instruction tuning teaches a plausible answer, but multiple plausible answers can differ in helpfulness, safety, accuracy, or teaching quality.

The Class 28 preference examples make this concrete. For “How should I prepare for exams?”, a chosen answer recommends scheduling, revision, practice, and sleep; a rejected answer encourages panic. For teaching skills, the preferred response explains step by step, uses examples, asks questions, and summarizes rather than speaking fast and ignoring learners.

Compare the approaches:

Approach	Core idea	Components
SFT	Imitate a desired response	prompt and target response
RLHF with PPO	Optimize a policy against a learned reward while constraining drift	policy, reference model, reward model, value model, PPO loop
DPO	Directly increase preference for chosen over rejected responses relative to a reference	policy/reference plus preference pairs

In the PPO workflow, a reward model first learns to score chosen responses above rejected ones. PPO then updates the policy using reward, a value estimate, and a KL penalty against a frozen reference model. DPO removes the explicit reward-model-and-RL loop and learns directly from paired preferences. It is operationally simpler, but still depends on representative, high-quality preference data.

Ask

“A company changes its refund policy every month. Should we fine-tune the model each month?”

Expected answer: usually no. Store changing facts externally and retrieve them. Fine-tune for behavior, style, specialized task patterns, or durable domain adaptation.

Transition

“That answer leads directly to RAG: keep changing knowledge outside the weights and provide the relevant evidence at request time.”

47:00–58:00 — 29–31: Retrieval-Augmented Generation

Teaching outcome

Learners should be able to explain, build, and troubleshoot the two halves of a RAG pipeline: indexing and question answering.

47:00–49:00 — Why RAG exists

Use the trainer’s refund-policy example:

```
User: What is the company refund policy?
Ungrounded model: Refunds are available within 30 days.
Company document: Refunds are available within 7 days.
Grounded answer: The policy allows refunds within 7 days.
```

RAG retrieves relevant external evidence and places it into the model’s prompt before generation. It is useful for private data, frequently changing information, source-grounded answers, and reducing unsupported claims.

Contrast it with fine-tuning:

Question	RAG	Fine-tuning
Where is new information stored?	External documents/index	Model parameters or adapters
Is an update easy?	Re-index changed content	Curate data and retrain
Primary goal	Supply current/private facts	Change behavior or durable patterns
Can it cite evidence?	Naturally, if metadata is preserved	Not inherently
Typical compute cost	Embedding, retrieval, inference	Training plus inference

Strong systems often combine them: a tuned model for behavior and a retriever for current evidence.

49:00–52:00 — Ingestion, parsing, and document structure

Draw the indexing half:

```
PDF / DOCX / HTML / JSON / image / audio
  ↓
loader/parser
  ↓
Document(page_content, metadata)
  ↓
chunking
  ↓
embeddings
  ↓
vector database
```

Class 30 used LangChain loaders for PDF, text, CSV, JSON, HTML, Word, Markdown, and web pages. Explain the wrapper’s value: different sources become a shared `Document` representation. LangChain is an integration layer; the underlying parser still determines extraction quality.

For simple text, a standard loader may be enough. Complex PDFs may contain reading-order problems, tables, embedded images, scanned pages, headers, and forms. The class used:

- PyMuPDF for text and embedded images.
- `pytesseract` for OCR on scanned image content.
- `pdfplumber` for table extraction.
- Docling for richer conversion to structured Markdown or JSON.

Metadata is not decoration. Preserve source file, page, section, content type, timestamp, access scope, and document ID so the system can filter, cite, update, and delete correctly.

52:00–54:00 — Chunking and embeddings

Chunking balances two failures:

- Chunks that are too large contain noise and consume context.
- Chunks that are too small lose surrounding meaning.

The class examples used recursive character splitting, including sizes around 1,000–1,200 characters and overlap around 200–400. These are starting values, not universal defaults. Choose boundaries using document structure and evaluate them with real questions.

Overlap helps preserve meaning across boundaries but increases index size and duplicate retrieval. Each chunk is embedded using the same embedding model used for the query. Embedding dimensions must match the vector index dimensions.

Vector-store examples introduced in the course include FAISS, Chroma, Qdrant, pgvector, Milvus, Pinecone, OpenSearch, Supabase, and Azure AI Search. Choose based on scale, filters, operations, security, hosting, and hybrid-search requirements—not simply popularity.

54:00–56:30 — Retrieval, prompt augmentation, and generation

Draw the online path:

```
question
  ↓ embed
query vector
  ↓ similarity search + optional filters
top-k chunks
  ↓ format with source metadata
grounded prompt
  ↓ LLM
answer + citations or abstention
```

Class 31 assembled the pipeline with `PyPDFLoader`, `RecursiveCharacterTextSplitter`, OpenAI embeddings, a FAISS index, a top-`k` retriever, a prompt template, a Gemini chat model, and LangChain runnable composition.

A compact form of the course prompt is:

```
Use only the supplied context to answer the question.
If the answer is not present, say that the context does not contain it.

Context:
{context}

Question:
{question}
```

This instruction does not guarantee grounding by itself. Retrieval quality determines whether the needed evidence reaches the model.

Use the DeepSeek V4 document example from the final class:

- V4-Pro was retrieved as a 1.6-trillion-parameter model with 49 billion active parameters per token.

- V4-Flash was retrieved as 284 billion total parameters with 13 billion active.
- The document associated reinforcement learning with GRPO.
- When the release date was not in the retrieved context, the chain answered that no information was available instead of inventing one.

The earlier ungrounded assistant claimed that V4 did not exist. This contrast is the closing proof: RAG can provide domain or newer evidence that the base model does not reliably know.

56:30–58:00 — Diagnose RAG as a pipeline

Teach learners not to blame every bad answer on the LLM. Use this diagnostic order:

1. **Parsing:** Was the needed information extracted correctly?
2. **Metadata and access:** Was the right document eligible for retrieval?
3. **Chunking:** Does one chunk retain enough evidence?
4. **Embedding:** Does the model represent this domain and language well?
5. **Retrieval:** Is the correct chunk in the top results?
6. **Prompt:** Is context clearly separated from instructions and the question?
7. **Generation:** Does the model use the evidence, cite it, and abstain when absent?
8. **Evaluation:** Are you measuring retrieval and answer quality with representative questions?

Relevant metrics include retrieval hit rate or recall at k , ranking quality, answer correctness, faithfulness to context, citation accuracy, latency, and cost.

Ask

"The correct answer exists in a PDF, but the model answers incorrectly. What should you inspect first?"

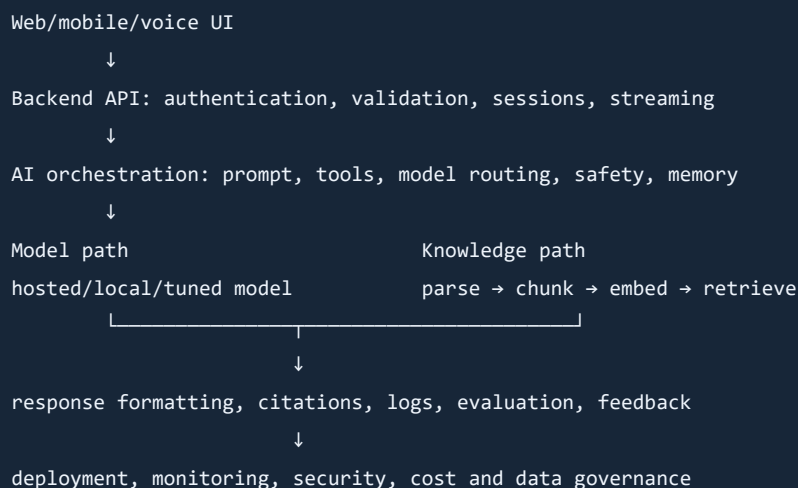
Expected answer: inspect the retrieved chunks. If the evidence was not retrieved, prompt tuning alone will not fix the root cause.

Transition

"We now have the model and knowledge layers. A full-stack trainer must connect them to the rest of the product."

58:00–60:00 — Full-stack close and teach-back

Draw the complete application once



Say:

"A model demo becomes a full-stack AI product only when it has a usable interface, a secure backend, dependable data and retrieval, observable model behavior, evaluation, and an operating plan."

The workspace teaches the AI/model pipeline in depth and includes a Streamlit application example. As an aspiring AI full-stack trainer, explicitly add the product bridge when you teach: frontend state, backend contracts, authentication, persistence, streaming, deployment, monitoring, evaluation, and security. Do not imply that calling an LLM API alone is a full-stack architecture.

Final teach-back

Ask one learner to complete this sentence in 30 seconds:

"My application uses _____ for behavior, _____ for changing knowledge, and exposes it through _____."

Example answer:

"My support application uses instruction tuning for support-answer style, RAG for current product policies, and exposes it through a secured backend API and chat UI."

Close with the course story:

"We began by representing data, learned how transformers create context, accessed and adapted foundation models, and ended by grounding those models in external knowledge. The trainer's job is to make those layers feel like one system."

Class-by-class coverage guide

Use this section to prepare a longer lesson, answer a learner who asks “Where did we learn that?”, or verify that the recap has not skipped a class. It is not intended to be read row by row during the 60-minute delivery.

Class	Main concept	Trainer example or artifact	One-sentence teaching point
1	Environment setup and GenAI introduction	Python, VS Code, <code>uv</code> , virtual environment	Reliable AI work begins with a reproducible development environment.
2	AI learning roadmap and Python foundations	<code>Calculator</code> class with safe division	AI applications combine probabilistic models with normal, testable software.
3	Encodings and embeddings	River bank vs financial bank	Encodings make data numeric; learned embeddings can place meaning into vector geometry.
4	One-hot encoding and Bag of Words	Sunny sentence; movie/cricket corpus	Sparse encodings preserve categories or counts but lose semantics and order.
5	TF-IDF and Word2Vec bridge	Same movie/cricket corpus	TF-IDF improves lexical weighting but does not create contextual meaning.
6–7	Pretrained and custom Word2Vec	<code>king - man + woman</code> ; Arjun/AI stories	Static word vectors learn useful relationships, but one word retains one vector and small training data is noisy.
8	State-of-the-art embeddings	Heart-risk semantic search; CLIP image vector	Contextual and multimodal embeddings enable meaning-based retrieval beyond exact keywords.
9	Transformer motivation	RNN/LSTM versus parallel attention	Transformers reduce sequential training bottlenecks and connect distant tokens directly.
10	Transformer depth	stacks, residual connection, FFN	Repeated blocks refine representations while residual paths preserve information and gradients.
11	Positional encoding and attention setup	<code>dog bites man</code> ; <code>money bank grows</code>	Position supplies order, and Q/K/V projections let tokens search for relevant context.
12	Self-attention derivation	<code>money bank grow</code> matrix calculation	Scaled query-key similarity becomes weights used to combine value vectors.
13	Multi-head attention and layer normalization	Chicken ambiguity; telescope ambiguity; <code>[0.8, -1.2, 2.4]</code>	Heads learn multiple relationship views while normalization stabilizes token features.
14	Decoder, masks, and cross-attention	<code>we are friends</code> translation	The decoder hides future targets and attends to encoder output while generating autoregressively.
15	LLM/SLM/multimodal categories and transformer code	German-to-English transformer from scratch	Model code connects tokenization, masking, attention, loss, training, and generation.
16–17	Hosted/local APIs and multimodal applications	Capital of India; language translation; cat image; image/audio examples; Streamlit	Provider wrappers and SDKs connect model capabilities to user-facing input and output flows.
18	Fine-tuning foundations	Pharma specialization pipeline	Continued pretraining, instruction tuning, and preference tuning solve different adaptation goals.
19	Hugging Face introduction and Unsloth orientation	Hub access and model/dataset repositories	The Hub organizes reusable model-development artifacts.
20	Hugging Face deep dive	IMDb transforms, BPE tokenizer, AutoModel task heads	Dataset schemas, tokenizers, and model heads must match the training objective.

Class	Main concept	Trainer example or artifact	One-sentence teaching point
21	Non-instruction fine-tuning	Raw pharma PDF packed into causal blocks	Continued pretraining teaches domain distribution, not necessarily instruction following.
22	Instruction fine-tuning	Metformin mechanism instruction/response	SFT teaches how to respond to a task using curated demonstrations.
23	Preference fine-tuning	Accurate chosen metformin answer vs false rejected answer	Preference pairs teach which response behavior is more desirable.
24	End-to-end Unsloth training	Three pharma stages and PEFT comparison	Efficient tooling makes multi-stage adapter training practical on constrained hardware.
25	Multimodal fine-tuning	Five iPhone images and one-sentence captions	Vision-language tuning requires aligned image, instruction, and response records.
26	Managed GPT fine-tuning	Smartphone warranty and app-availability JSONL	Managed services simplify infrastructure, but data validation and evaluation remain the developer's job.
27	LoRA and QLoRA	TinyLlama adapters targeting attention/MLP projections	Low-rank adapters and quantized bases reduce trainable parameters and memory.
28	RLHF, reward models, PPO, and DPO	Preferred exam, fitness, AI, and teaching answers	Alignment optimizes preference after basic response imitation.
29	RAG foundations	30-day hallucination vs documented 7-day refund	RAG injects relevant external knowledge without changing model weights.
30	Data loading and complex parsing	multi-format loaders; PDF tables, images, OCR, Docling	Retrieval cannot recover information that parsing lost or stripped of structure.
31	End-to-end RAG	DeepSeek V4 PDF, FAISS, embeddings, retriever, grounded chain	A complete RAG pipeline must retrieve evidence, constrain generation, and abstain when evidence is absent.

Trainer reference sheets

Representation decision guide

Need	Start with	Reason
Small categorical feature for a conventional ML model	One-hot encoding	Simple and explicit categories
Basic lexical document baseline	BoW or TF-IDF	Cheap, interpretable, and easy to debug
Word similarity with lightweight static vectors	Word2Vec/FastText	Dense semantic word representations
Sentence or paragraph semantic search	Transformer embedding model	Contextual meaning and fixed-size vectors
Cross-modal image-text similarity	CLIP-like multimodal embedding	Places images and text in a comparable space
Private/current document answers	Embeddings plus retrieval/RAG	Evidence remains outside model weights and is updatable

Model adaptation decision guide

Ask these questions in order:

1. Is the problem missing or changing information? Start with RAG.
2. Is the problem output format, tone, task behavior, or domain reasoning? Consider SFT.
3. Is raw domain language missing from the base model? Consider continued pretraining, then SFT.
4. Are several answers acceptable but one is preferred? Consider preference alignment.
5. Is GPU memory limited? Prefer LoRA or QLoRA over full fine-tuning.
6. Is the dataset large, representative, licensed, private, and clean enough to justify training?
7. Do offline and online evaluations show that prompt/RAG changes are insufficient?

Core formula board

Cosine similarity

$$\text{cosine}(\mathbf{a}, \mathbf{b}) = (\mathbf{a} \cdot \mathbf{b}) / (||\mathbf{a}|| \ ||\mathbf{b}||)$$

It compares direction rather than raw magnitude.

Scaled dot-product attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

Queries score keys; normalized scores weight values.

Sinusoidal positional encoding

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

It injects sequence order without changing the embedding width.

Layer normalization

$$\hat{x} = (x - \text{mean}(x)) / \sqrt{\text{variance}(x) + \epsilon}$$
$$y = \gamma \hat{x} + \beta$$

It normalizes each token's feature vector and then learns a scale and shift.

LoRA

$$W' = W + (\alpha/r)BA$$

The base weight remains frozen while low-rank matrices learn the update.

RAG minimal pseudocode

Keep the indexing and answering paths visibly separate:

```
# Offline or update-time indexing
documents = load_sources()
chunks = split_documents(documents)
vectors = embedding_model.embed_documents([c.page_content for c in chunks])
vector_store.add(chunks, vectors)

# Online question answering
query_vector = embedding_model.embed_query(question)
contexts = vector_store.search(query_vector, k=5)
prompt = grounded_prompt(contexts, question)
answer = chat_model.invoke(prompt)
```

Production additions include access control, hybrid retrieval, reranking, deduplication, citations, prompt-injection defenses, evaluation, caching, monitoring, and feedback.

AI full-stack trainer bridge

The class sequence is strongest on representations, transformers, model access, adaptation, and RAG. To teach “AI full stack” confidently, connect every model lesson to these software layers.

Layer	What to teach	Questions learners should answer
Frontend	chat state, file upload, streaming tokens, citations, errors, accessibility	What does the user see while inference is running or fails?
Backend API	request schema, validation, auth, rate limits, async work, cancellation	What is the contract between UI and AI service?
Orchestration	prompts, tools, model routing, memory, retries, safety	Which steps are deterministic and which use a model?
Model	hosted vs local, base vs tuned, latency, token limits	Why is this model appropriate for this task?
Knowledge	parsing, chunking, embeddings, retrieval, metadata, permissions	Can the right evidence be retrieved and cited securely?
Data/state	user sessions, conversation state, feedback, audit records	What persists, for how long, and for whom?
Evaluation	golden questions, retrieval metrics, groundedness, safety, regression tests	How will a release be declared better?
Operations	deployment, secrets, logs, traces, cost, scaling, fallbacks	What happens under load or provider failure?
Governance	privacy, licensing, retention, access control, human approval	What data may enter the model or training set?

A reusable full-stack classroom case study

Use one case study across future lessons instead of changing domains every class:

Product: a smartphone support assistant.

```

React/Streamlit chat and image upload
→ FastAPI request validation and authenticated session
→ router selects text or vision model
→ RAG retrieves current warranty and app-support policies
→ optional tuned model follows the company's support style
→ response includes answer, source, confidence/abstention, and next action
→ traces and learner feedback feed evaluation dashboards

```

This one product lets you demonstrate:

- Encodings and embeddings through policy search.
- Transformers through contextual questions.
- Multimodality through phone screenshots.
- Fine-tuning through support tone and response format.
- RAG through frequently changing warranty documents.
- Tool use through warranty lookup or ticket creation.
- Full-stack concerns through sessions, APIs, streaming, auth, deployment, and monitoring.

The trainer's five-question design review

Before showing any AI application, ask:

1. What user problem are we solving, and what does a correct result look like?
 2. Which information belongs in model weights, retrieved context, application state, or deterministic tools?
 3. What can fail at the UI, API, retrieval, model, or provider layer?
 4. How will we evaluate correctness, grounding, safety, latency, and cost?
 5. What evidence will make us comfortable deploying the next version?
-

Delivery checks and common misconceptions

Correct these statements during teaching

Misconception	Better explanation
"An embedding is just any numeric encoding."	Embeddings are learned dense representations intended to preserve useful relationships; encoding is the broader category.
"TF-IDF understands meaning."	It is a powerful lexical weighting baseline, but it primarily uses term occurrence statistics.
"Word2Vec understands every meaning of a word."	Classical Word2Vec normally assigns one static vector to each vocabulary item.
"Transformers process everything in parallel."	Training can parallelize positions, but autoregressive inference still generates sequentially.
"Attention tells us exactly how the model reasons."	Attention weights are internal interactions, not a guaranteed faithful explanation of reasoning.
"More layers or parameters always make a model better."	Quality depends on data, objective, architecture, training, evaluation, latency, and task fit.
"Temperature controls factual accuracy."	It primarily affects sampling variability; grounding and evidence are separate concerns.
"Fine-tuning is how we upload documents."	Use RAG for changing or source-grounded knowledge; fine-tuning changes model behavior or learned patterns.
"QLoRA quantizes the trainable adapter to four bits."	In the usual course setup the frozen base is quantized, while LoRA adapters are trained with higher-precision computation.
"A successful training job means a better model."	It means optimization completed; held-out and task-level evaluation must prove improvement.
"RAG prevents hallucination."	RAG supplies evidence, but bad parsing, retrieval, prompts, or generation can still produce unsupported answers.
"A vector database is the whole RAG system."	It is one component between ingestion and grounded generation.
"An LLM API call is a full-stack application."	A product also needs UI, backend, security, state, evaluation, operations, and governance.

Signs that the recap is working

- Learners can explain the limitations that led from OHE to transformer embeddings.
- They can trace **bank** through query, key, value, attention weights, and a contextual output.
- They can distinguish causal masking from cross-attention.
- They choose RAG for changing policy facts and SFT for response behavior.
- They can explain why QLoRA reduces memory.
- They inspect retrieved chunks before rewriting the RAG prompt.
- They describe an AI feature as a complete application path, not only a model call.

If you are running behind

Preserve the story, not every library name. Shorten in this order:

1. Mention fewer API providers in lessons 16–17.
2. Describe the Hugging Face mechanics without opening the Hub.
3. Explain only one multimodal fine-tuning example.

4. Keep PPO/DPO at comparison level.

Do **not** cut the embedding evolution, Q/K/V intuition, RAG versus fine-tuning decision, or the final full-stack diagram. Those are the conceptual spine of the course.

If you have two extra minutes after questions

Ask learners to diagnose this scenario:

"A support chatbot gives a fluent but outdated warranty answer even though the latest policy is in the document store."

A strong answer checks document ingestion and versioning, permissions and metadata filters, chunking, query embedding, retrieved top-**k** chunks, grounded prompt construction, and response citations before changing the model.

Final trainer preparation checklist

- ☐ I can deliver the seven timed blocks in exactly 60 minutes.
- ☐ I can draw the representation evolution without notes.
- ☐ I can derive scaled dot-product attention in plain language.
- ☐ I can explain encoder-only, decoder-only, and encoder-decoder use cases.
- ☐ I have two small pre-tested demos and screenshot fallbacks for larger ones.
- ☐ I can compare RAG, SFT, preference tuning, LoRA, and QLoRA by problem—not by fashion.
- ☐ I can trace a RAG failure from parser to answer.
- ☐ I can place the AI pipeline behind a secured backend and user interface.
- ☐ I can state what the current notebooks demonstrate and what would still be required for production.
- ☐ I end with learner teach-back rather than another lecture slide.

This is the core trainer message to retain:

Represent the data well, understand the model mechanism, adapt only when the problem requires it, retrieve changing evidence, and engineer the entire application around the model.